Chapter 1: Software and Software Engineering

Definitions

Software and engineering

- **Software:** Programs and associated data and documentation that enable a computer system to perform tasks.
- **Software engineering:** Systematic development and evolution of sufficiently high-quality software that solves customer problems within cost, time, and other constraints.
- **Software evolution:** Ongoing modification of software as needs, technology, and understanding change.
- **Real-time system:** A system whose correctness depends on producing results within required time limits as well as producing correct results.
- **Embedded system:** A computer system integrated into a larger device to monitor or control its operation.
- **Data-processing system:** Software that collects, stores, transforms, and reports data to support operations such as business transactions.
- **Mobile application:** Software designed to run on mobile devices such as phones or tablets.
- **Web application:** Software whose functionality users access through web technologies, commonly through a browser.
- **Professional ethics:** Principles guiding responsible conduct toward the public, clients, employers, colleagues, and the profession.
- **Constraint:** A limit or condition that a solution or its development must satisfy.

Quality and stakeholders

- **Software quality:** The extent to which software satisfies required behaviour and the needs and quality objectives of its stakeholders.
- **Usability:** How effectively users can learn and use a system to accomplish their tasks.
- **Efficiency:** How economically software uses processor time, memory, and other resources to perform its work.
- **Reliability:** The ability to perform required functions without failure under stated conditions for a specified period.
- **Maintainability:** The ease with which developers can understand, correct, and modify software.

- **Reusability:** The extent to which software components can be used in other applications or projects.
- **Stakeholder:** A person or organization affected by a system or having an interest in its development or operation.
- **User:** A person who interacts with the system to accomplish tasks.
- **Customer:** The person or organization requesting or purchasing a solution to a problem.
- **Developer:** A person who contributes to creating or changing software.
- **Development manager:** A person who coordinates development work, resources, costs, and delivery objectives.
- **Quality tradeoff:** A decision balancing competing quality goals, such as performance and maintainability.
- **Over-engineering:** Adding complexity or capabilities beyond what the problem and quality objectives justify.
- **Defect:** A flaw in a software artifact that can cause incorrect behaviour or failure to meet a requirement.
- **Failure:** An observed inability of a system to perform as required.

Projects and maintenance

- **Software maintenance:** Modification of existing software to correct problems, adapt to changes, or improve it.
- **Corrective maintenance:** Changes that fix defects.
- **Adaptive maintenance:** Changes that keep software usable when its environment, platforms, or external rules change.
- **Enhancement:** A change that adds or extends user-visible capabilities.
- **Reengineering:** Examining and restructuring existing software to improve its form and make continued evolution easier.
- **Perfective maintenance:** Changes that improve existing software, including its performance or maintainability; the lecture emphasizes internal structural improvements.
- **Greenfield development:** Creating a new system rather than modifying an existing one.
- **Legacy system:** An existing system inherited from earlier development that remains in use and must often be maintained or evolved.

Development activities and models

- **Domain:** The business or technical field in which a system operates.

- **Domain analysis:** Learning about that field and the problem the software should address.
- **Requirement:** A statement of required system behaviour or a constraint on the system or its development.
- **Requirements gathering:** Obtaining information about needs and constraints from stakeholders and other sources.
- **Requirements analysis:** Organizing, examining, and resolving gathered information to establish coherent requirements.
- **Specification:** A documented description of required behaviour and constraints.
- **Design:** Deciding how a system will be organized and how its parts will satisfy requirements.
- **Systems engineering:** Coordinating the overall system design, including allocating responsibilities between hardware and software.
- **Software architecture:** The high-level organization of software into subsystems and their interactions.
- **Subsystem:** A cooperating part of a larger system with a defined set of responsibilities.
- **Detailed design:** Decisions about subsystem internals, algorithms, user interfaces, and data structures.
- **Model:** A simplified representation emphasizing selected aspects of a system.
- **Use-case model:** A model of actor goals and the interactions through which the system supports those goals.
- **Structural model:** A model showing a system's elements and how they are organized or related.
- **Behavioural model:** A model showing actions, interactions, and changes over time.
- **Implementation / programming:** Translating a design into executable software.
- **Review:** Examination of a software artifact to assess quality and find problems.
- **Inspection:** A structured review using a defined process to detect defects in an artifact.
- **Testing:** Executing software under selected conditions and comparing observed behaviour with expected results.
- **Deployment:** Making software available for use in its intended environment.
- **Process management:** Planning and coordinating development activities, resources, and progress.

1.0 The Nature of Software

- Software is intangible: effort, progress, and quality can be difficult to see.
- Reproduction is inexpensive compared with development. Development is labour-intensive and hard to automate completely.
- Easy modification can hide expensive consequences: changes made without understanding the design can introduce defects and complexity.
- Software does not physically wear out. Its structure can deteriorate through poorly planned changes.

1.0.0 Types of software

- Real-time embedded systems monitor or control equipment and must meet timing requirements; safety may be critical.
- Data-processing systems support business operations, where data accuracy and security matter.
- Games, mobile applications, and web applications have different users, operating environments, and quality priorities.

1.1 What is Software Engineering?

Software engineering solves customer problems through systematic development and evolution of large, sufficiently high-quality software systems within cost, time, and other constraints.

- Understand the customer's problem before choosing a solution. Buying or adapting existing software may be appropriate.
- Apply established techniques in an organized way. Much development evolves existing systems.
- Divide work across a team and coordinate the interfaces between parts. One person cannot understand every detail of a large system.
- Estimate resources realistically and ensure benefits justify costs. Extra features can increase complexity without solving the problem.

1.2 The Software Engineering Profession

The term software engineering dates to 1968. An engineering approach combines technical knowledge, disciplined practice, and responsibility for the resulting systems.

1.2.0 Ethics

- Protect the public interest and act responsibly toward clients and employers.

- Aim for high product and maintenance standards, and preserve independent professional judgment.
- Promote ethical management and the integrity of the profession.
- Support colleagues fairly and continue learning throughout a career.

1.3 Software Quality

- Usability: users can learn the system and complete their work effectively.
- Efficiency: the system avoids unnecessary use of processor time, memory, and other resources.
- Reliability: the system performs required functions without failure under stated conditions.
- Maintainability: developers can understand and change the system without excessive effort.
- Reusability: components can serve other projects or applications.

1.3.0 Stakeholders and tradeoffs

- Users value ease of learning and effective task completion; customers value solving a problem at an acceptable total cost.
- Developers value understandable designs and components that are easy to maintain and reuse.
- Development managers balance customer satisfaction with development and maintenance costs.
- Quality goals can conflict. Optimization may make code harder to maintain; a richer interface may consume more resources.
- Set quality objectives before designing. Over-engineering beyond those needs wastes resources.

1.4 Software Engineering Projects

- Corrective maintenance fixes defects.
- Adaptive maintenance responds to environmental changes such as operating systems, databases, or external rules.
- Enhancement adds user-visible capabilities.
- Reengineering or perfective work improves internal structure and maintainability.
- Greenfield development creates a new system. Many projects instead evolve legacy systems.

1.5 Activities Common to Software Projects

1.5.0 Requirements and specification

- Domain analysis develops understanding of the field and problem.
- Gathering obtains information from stakeholders and other sources.
- Analysis organizes and resolves that information. Specification records required behaviour and constraints.

1.5.1 Design

- Systems engineering allocates responsibilities between hardware and software.
- Architecture divides software into subsystems and defines their interactions.
- Detailed design determines subsystem internals, user interfaces, and database structures.

1.5.2 Modelling, implementation, and delivery

- Use-case models describe goals and interactions. Structural models describe organization. Behavioural models describe interactions and changes over time.
- Programming implements the design. Reviews, inspections, and tests assess quality.
- Deployment makes the system available. Process management coordinates work, resources, and progress.
- These activities recur and inform one another as software evolves.